



Gestor de gastos: listado de gastos

Este caso práctico es continuación del anterior, por lo que seguiremos utilizando el mismo repositorio de código. Para esta práctica, los test se ejecutan con `npm run test2`. De nuevo, si se está utilizando Visual Studio Code se pueden ejecutar los test desde el propio editor.

Utilizaremos de nuevo el fichero `js/gestionPresupuesto.js`. A no ser que se indique lo contrario, todo el código que se cree deberá guardarse en este fichero.

De momento, nuestra librería es capaz de crear objetos **gasto** básicos, con una descripción y un valor. Dichos objetos disponen también de métodos para actualizar sus propiedades.

En esta práctica, mejoraremos los objetos **gasto**, añadiéndoles propiedades que almacenen la **fecha de creación** y un **listado de etiquetas** para categorizarlos. Además, añadiremos la funcionalidad para **gestionar la lista de gastos**. Para ello, necesitaremos almacenar dicha lista en un **array** y definir una serie de funciones en nuestra librería para **añadir y eliminar gastos** de dicha lista.

Las tareas que hay que realizar son las siguientes:

- Añade las siguientes **variables globales**:
 - **gastos**. Almacenará el **listado de gastos** que vaya introduciendo el usuario. **Inicialmente** contendrá un **array vacío**.
 - **idGasto**. Se utilizará para almacenar el **identificador actual** de cada gasto que se vaya añadiendo. Su **valor inicial** será **0**. Se irá incrementando con cada gasto que se añada.
- Añade las funciones **listarGastos**, **anyadirGasto**, **borrarGasto**, **calcularTotalGastos** y **calcularBalance** al objeto **export** del final del fichero.
- Define las funciones vacías (sin parámetros y sin cuerpo) en el fichero, para que los test no den error de sintaxis y se puedan ir comprobando conforme se vaya avanzando en la práctica.
- Crea una función **listarGastos**. Será una función **sin parámetros**, que devolverá la variable global **gastos**. La variable **gastos** es global, pero solo dentro del módulo: si queremos que el programa que utilice nuestra librería pueda visualizar el listado de gastos, tendremos que **exportar** una función que devuelva su contenido.
- **Actualiza** la función constructora **CrearGasto** para que incluya la **fecha y las etiquetas** (más detalles, en la descripción del objeto **gasto**). Los parámetros adicionales de la función deben ir a continuación de los existentes.
 - Si no se indican los parámetros de **etiquetas**, se almacenará en la propiedad **etiquetas** un **array vacío**.
 - Si no se indica el parámetro **fecha**, se almacenará en la propiedad **fecha** la **fecha actual**.
 - El parámetro **fecha** deberá ser un **string** con formato válido que pueda entender la función **Date.parse**. Si la fecha no es válida (no sigue el formato indicado), se deberá almacenar la **fecha actual** en su lugar.
 - Tal como se indica en la sección de objeto **gasto**, la fecha se almacenará en formato **timestamp**.
 - Las etiquetas se pasarán como una lista de parámetros de número indeterminado.
 - Para añadir las etiquetas, se utilizará el método **anyadirEtiquetas**, explicado en la sección de objeto **gasto**. Algunos ejemplos de llamadas de función **CrearGasto** podrían ser:

```
let gasto1 = new CrearGasto("Gasto 1");
let gasto2 = new CrearGasto("Gasto 2", 23.55);
let gasto3 = new CrearGasto("Gasto 3", 23.55, "2021-10-06T13:10");
let gasto4 = new CrearGasto("Gasto 4", 23.55, "2021-10-06T13:10", "casa");
let gasto5 = new CrearGasto("Gasto 5", 23.55, "2021-10-06T13:10", "casa",
"supermercado");
let gasto6 = new CrearGasto("Gasto 6", 23.55, "2021-10-06T13:10", "casa",
"supermercado", "comida");
```

- Crea una función **anyadirGasto**. Será una función de un **parámetro** (un objeto de tipo **gasto**) y realizará tres tareas:
 - Añadir al objeto **gasto** pasado como parámetro una propiedad **id** cuyo valor será el valor actual de la variable global **idGasto**.
 - Incrementar el valor de la variable global **idGasto**.
 - Añadir el objeto **gasto** pasado como parámetro a la variable global **gastos**. El gasto se debe añadir al final del *array*.
- Crea una función **borrarGasto**. Será una función de un **parámetro**, que eliminará de la variable global **gastos** el objeto **gasto** cuyo **id** haya sido pasado como parámetro. Si no existe un gasto con el **id** proporcionado, no hará nada.
- Crea una función **calcularTotalGastos**. Será una función sin **parámetros**, que devolverá la suma de todos los gastos creados en la variable global **gastos**.
- Crea una función **calcularBalance**. Será una función sin **parámetros**, que devolverá el balance (presupuesto – gastos totales) disponible.
- Los objetos **gasto** creados por la función constructora **CrearGasto** deberán tener las siguientes propiedades adicionales:
 - **fecha**. Almacenará la fecha en que se crea el gasto en forma de *timestamp* (consulta para ello el enlace de *Gestión de fecha y hora*, de la documentación adicional).
 - **etiquetas**. Almacenará en un *array* el listado de etiquetas (categorías) asociadas al gasto.
 - **mostrarGastoCompleto**. Función sin **parámetros**, que devolverá el texto multilinea siguiente (ejemplo para un gasto con tres etiquetas):

Gasto correspondiente a DESCRIPCION con valor VALOR €.
Fecha: FECHA_EN_FORMATO_LOCALIZADO
Etiquetas:
 - ETIQUETA 1
 - ETIQUETA 2
 - ETIQUETA 3

Para mostrar la fecha en formato localizado, puedes utilizar el método `toLocaleString()` (consulta para ello el enlace de *Gestión de fecha y hora*, de la documentación adicional).

- **actualizarFecha**. Función de un **parámetro**, que actualizará la propiedad **fecha** del objeto. Deberá recibir la fecha en formato **string** que sea entendible por la función `Date.parse`. Si la fecha no es válida, se dejará sin modificar.
- **anyadirEtiquetas**. Función de un número indeterminado de **parámetros**, que añadirá las etiquetas pasadas como parámetro a la propiedad **etiquetas** del objeto. Deberá comprobar que no se creen duplicados.
- **borrarEtiquetas**. Función de un número indeterminado de **parámetros**, que recibirá uno o varios nombres de etiquetas y procederá a eliminarlas (si existen) de la propiedad **etiquetas** del objeto.

Claves de resolución

En primer lugar, comprueba que pasas los test de la práctica anterior. Si no es el caso, completa esa parte primero, antes de proseguir.

Dedica un tiempo a familiarizarte con el enunciado y todas las tareas que se piden. Inspecciona también el archivo del segundo test para ver cómo funciona.

Por último, empieza a escribir el código que se pide en el enunciado. Cada vez que creas haber completado una de las tareas que se piden, lanza los test mediante `npm run test2` y comprueba si superas alguno nuevo. Ve realizando *commits* en el repositorio de manera periódica; sobre todo, cuando consigas superar un nuevo test.